

SQL CASE Statements & CASE Expressions

Scenario-based MySQL assignment • No solutions

Goal: Practice simple CASE, searched CASE, conditional aggregation, custom sorting, and CASE combined with window functions.

Quick Revision

Concept	Pattern
Searched CASE	CASE WHEN condition THEN result ELSE result END
Simple CASE	CASE expression WHEN value THEN result ELSE result END
Conditional count	SUM(CASE WHEN condition THEN 1 ELSE 0 END)
Conditional total	SUM(CASE WHEN condition THEN amount ELSE 0 END)

Scenario 1 — E-Commerce Orders

Use CASE to classify orders, calculate conditional revenue, and build customer-level reports.

Schema

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY,  
  customer_name VARCHAR(50),  
  city VARCHAR(50),  
  customer_type VARCHAR(20)  
);  
  
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_id INT,  
  order_date DATE,  
  order_value DECIMAL(10,2),  
  payment_method VARCHAR(20),  
  order_status VARCHAR(20),  
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

customers

ID	Name	City	Type
1	Rahul	Mumbai	Regular
2	Priya	Pune	Premium
3	Amit	Nagpur	Regular
4	Sneha	Mumbai	Premium
5	Karan	Pune	Regular
6	Neha	Nagpur	Premium

orders

ID	Cust.	Date	Value	Payment	Status
101	1	2026-01-05	5000	UPI	Delivered
102	2	2026-01-08	7000	Card	Delivered
103	1	2026-01-15	3000	UPI	Cancelled
104	3	2026-01-20	9000	Card	Delivered
105	4	2026-01-25	4000	COD	Pending
106	2	2026-02-03	5000	UPI	Delivered

ID	Cust.	Date	Value	Payment	Status
107	5	2026-02-10	8000	Card	Delivered
108	3	2026-02-15	6000	UPI	Returned
109	1	2026-02-20	7000	Card	Delivered
110	6	2026-02-25	5000	UPI	Delivered
111	4	2026-03-02	6000	Card	Delivered
112	5	2026-03-10	4000	COD	Pending

Part A — Basic to Intermediate CASE

Q1. Classify order value as High (≥ 7000), Medium (≥ 4000 and < 7000), or Low (< 4000) using searched CASE.

Q2. Using simple CASE, convert UPI $\rightarrow$ Digital UPI, Card $\rightarrow$ Credit/Debit Card, COD $\rightarrow$ Cash on Delivery.

Q3. Create a status label: Delivered $\rightarrow$ Completed, Pending $\rightarrow$ Open, Cancelled/Returned $\rightarrow$ Problematic.

Q4. Calculate total revenue from only Delivered orders using SUM(CASE...).

Q5. Calculate Delivered, Pending, Cancelled, and Returned value in separate columns.

Q6. Count Delivered, Pending, Cancelled, and Returned orders using conditional aggregation.

Q7. For every customer, show total order value and label them VIP (≥ 12000), Regular (≥ 7000), or Low Spender.

Q8. Find cities with at least 2 Delivered orders using conditional aggregation and HAVING.

Q9. Create a custom status priority: Pending=1, Delivered=2, Returned=3, Cancelled=4, then sort by priority.

Q10. Create a customer report with total orders, delivered orders, cancelled/returned orders, and a Clean/Review CASE label.

Scenario 2 — Logistics & Delivery

Schema

```
CREATE TABLE drivers (  
  driver_id INT PRIMARY KEY,  
  driver_name VARCHAR(50),  
  city VARCHAR(50)  
);  
  
CREATE TABLE deliveries (  
  delivery_id INT PRIMARY KEY,  
  driver_id INT,  
  delivery_date DATE,  
  delivery_time_hours DECIMAL(5,2),  
  delivery_value DECIMAL(10,2),  
  status VARCHAR(20),  
  FOREIGN KEY (driver_id) REFERENCES drivers(driver_id)  
);
```

drivers

ID	Driver	City
1	Ramesh	Mumbai
2	Suresh	Pune
3	Akash	Nagpur
4	Vikram	Mumbai
5	Rohit	Pune
6	Anil	Nagpur

deliveries

ID	Driver	Date	Hours	Value	Status
201	1	2026-01-05	4.5	5000	Delivered
202	2	2026-01-06	6.0	7000	Delayed
203	1	2026-01-10	3.5	4000	Delivered
204	3	2026-01-12	5.0	8000	Delivered
205	4	2026-01-15	7.0	6000	Delayed
206	2	2026-01-18	4.0	5000	Delivered
207	5	2026-01-20	5.5	9000	Delivered
208	3	2026-01-22	4.0	6000	Delivered
209	1	2026-01-25	6.0	7000	Delayed
210	6	2026-01-28	3.0	5000	Delivered
211	4	2026-02-02	5.5	7000	Delivered
212	5	2026-02-05	4.5	6000	Delayed

Part B — Logistics CASE Practice

Q11. Classify delivery time as Fast (<4 hours), Normal (4–5.5 hours), or Slow (>5.5 hours).

Q12. Classify delivery value as High (≥7000), Medium (≥5000), or Low (<5000).

Q13. Create a status label: Delayed → Needs Attention; Delivered → On Time; otherwise Unknown.

Q14. Count delayed deliveries for each driver using SUM(CASE...).

Q15. Calculate delayed-delivery percentage for each driver.

Q16. Calculate total delivery value and delayed delivery value for each city.

Q17. Classify drivers from average delivery time: Excellent ≤ 4 , Good ≤ 5.5 , Needs Improvement > 5.5 .

Q18. Using CASE inside SUM, calculate Fast-delivery value and Slow-delivery value for each driver.

Q19. Create a custom status priority and sort deliveries by priority, then delivery date.

Q20. Using a subquery or CTE, calculate each driver's total delivery value and label them Top Performer ≥ 14000 , Strong ≥ 10000 , Developing otherwise.

Scenario 3 — Warehouse Inventory

Schema

```
CREATE TABLE inventory (  
  inventory_id INT PRIMARY KEY,  
  warehouse VARCHAR(50),  
  product VARCHAR(50),  
  stock INT,  
  stock_value DECIMAL(12,2),  
  last_updated DATE  
);
```

inventory

ID	Warehouse	Product	Stock	Stock Value	Updated
301	Mumbai	Laptop	20	1200000	2026-01-05
302	Mumbai	Phone	50	750000	2026-01-06
303	Mumbai	Tablet	30	450000	2026-01-07
304	Pune	Laptop	15	900000	2026-01-08
305	Pune	Phone	60	900000	2026-01-09
306	Pune	Tablet	25	375000	2026-01-10
307	Nagpur	Laptop	10	600000	2026-01-11
308	Nagpur	Phone	40	600000	2026-01-12
309	Nagpur	Tablet	20	300000	2026-01-13
310	Mumbai	Monitor	25	500000	2026-01-14
311	Pune	Monitor	30	600000	2026-01-15
312	Nagpur	Monitor	15	300000	2026-01-16

Part C — Inventory & Conditional Aggregation

Q21. Classify stock as Low (<15), Medium (15–30), or High (>30).

Q22. Classify stock value as Low (<500000), Medium (500000–800000), or High (>800000).

Q23. Create stock_action: Low → Reorder, Medium → Monitor, High → No Action.

Q24. For each warehouse, count Low Stock products using SUM(CASE...).

Q25. For each warehouse, calculate stock value belonging to products with stock <20.

Q26. Calculate each warehouse's total stock value and percentage contributed by Low Stock products.

Q27. Using CASE with a window function, label products as Warehouse Risk when stock <20 and stock_value >500000.

Q28. Create a warehouse report with total products, low-stock products, high-stock products, and health: Critical if low-stock count >=2, Watch if =1, Healthy otherwise.

Q29. Create custom product priority: Laptop=1, Phone=2, Tablet=3, Monitor=4 and sort within each warehouse.

Q30. Final inventory challenge: combine CASE, SUM(), AVG(), and a window function to show warehouse, product, stock, stock value, warehouse total value, stock-risk label, and product contribution percentage.

Scenario 4 — Advanced CASE + Window Functions

Use the products table below. This section connects CASE with FIRST_VALUE, LAST_VALUE, NTH_VALUE, NTILE, and CUME_DIST.

Schema

```
CREATE TABLE products (  
  product_id INT PRIMARY KEY,  
  product_category VARCHAR(50),  
  brand VARCHAR(50),  
  product_name VARCHAR(100),  
  price DECIMAL(10,2)  
);
```

products

ID	Category	Brand	Product	Price
1	Laptop	Dell	Inspiron 15	65000
2	Laptop	HP	Pavilion 14	62000
3	Laptop	Lenovo	IdeaPad Slim 5	58000
4	Laptop	Dell	Vostro 15	72000
5	Laptop	HP	Victus 15	85000
6	Mobile	Samsung	Galaxy A55	42000
7	Mobile	Apple	iPhone 15	70000
8	Mobile	OnePlus	OnePlus 13R	45000
9	Mobile	Samsung	Galaxy S24	75000
10	Mobile	Apple	iPhone 15 Pro	125000
11	Tablet	Samsung	Galaxy Tab S9	55000
12	Tablet	Apple	iPad Air	60000
13	Tablet	Lenovo	Tab P12	35000
14	Tablet	Samsung	Galaxy Tab A9	18000
15	Tablet	Apple	iPad 10th Gen	45000
16	Headphones	Sony	WH-1000XM5	30000
17	Headphones	JBL	Tune 770NC	8500
18	Headphones	Boat	Rockerz 550	2500
19	Headphones	Sony	WH-CH720N	12000
20	Headphones	JBL	Live 660NC	15000
21	Monitor	LG	UltraGear 27	28000
22	Monitor	Samsung	Odyssey G5	32000
23	Monitor	Dell	S2721D	25000
24	Monitor	LG	UltraWide 29	35000
25	Monitor	Samsung	ViewFinity S6	40000

Part D — Advanced CASE + Window Functions

Q31. Use simple CASE to map Laptop → Computing, Mobile → Phones, Tablet → Computing, Headphones → Audio, Monitor → Display.

Q32. Use searched CASE to classify prices into Budget (<20000), Mid Range (20000–59999), Premium (60000–99999), Flagship (≥100000).

Q33. For every product, show its category's cheapest price using `FIRST_VALUE()` and a CASE label indicating whether the current product is cheapest.

Q34. For every category, use `LAST_VALUE()` to find the most expensive product and label it Category Leader using CASE.

Q35. Using `NTH_VALUE()` for the 2nd most expensive product, create a CASE column showing whether each product is above, equal to, or below the 2nd most expensive price.

Q36. Use `NTILE(4)` ordered by price DESC and CASE to label buckets as Top Quartile, Upper-Mid, Lower-Mid, and Bottom Quartile.

Q37. Use `CUME_DIST()` and CASE to label products as Top 25%, 25–50%, 50–75%, or Bottom 25% within each category.

Q38. Using `SUM(CASE...)`, count Premium/Flagship products for each brand.

Q39. Find brands having at least one product priced ≥ 100000 using HAVING with conditional aggregation.

Q40. Final challenge: create a complete pricing report containing category, brand, product, price, cheapest price, most expensive price, 3rd most expensive product, `NTILE(4)` bucket, `CUME_DIST()`, and a CASE-based price segment.

Bonus Interview Questions

B1. Explain simple CASE vs searched CASE with one example of each.

B2. Why does CASE stop after the first true WHEN?

B3. Why is `SUM(CASE WHEN condition THEN 1 ELSE 0 END)` useful for conditional counting?

B4. What is the difference between CASE in SELECT and CASE inside an aggregate?

B5. Write a query using CASE in ORDER BY for a custom sort.

B6. Write a query using CASE + a window function to label products Above, Equal to, or Below Category Average.

Practice tip: For every question, first identify the condition, then the returned value, then decide whether you need simple CASE or searched CASE. For reporting problems, look for `SUM(CASE...)`, `COUNT(CASE...)`, GROUP BY, HAVING, and window functions.